

Strategy Design Pattern (Behavioral)

Encapsulate each algorithm behind a common interface so that the algorithm used by a client can be swapped at runtime without modifying the client.

Intent

The Strategy pattern solves the problem of hard-coding a single algorithm inside a class, which forces a code change every time a different algorithm is needed. It defines a family of algorithms, encapsulates each one in its own class, and makes them interchangeable via a shared interface. The object that uses the algorithm (the *context*) delegates the work to whichever strategy it holds, and neither the context nor the caller needs to know the implementation details.

Real-world analogy

Think of a pocket calculator with a mode dial. The display, the number buttons, and the equals key never change — that is the **context**. What changes is the *operation* selected: add, subtract, multiply, or divide. Each mode is an independent, interchangeable strategy. Rotating the dial (injecting a different strategy) does not require rewiring the calculator; it simply changes which algorithm the fixed infrastructure delegates to when the equals key is pressed.

How the pattern works

When `main()` constructs a `calculator` it passes in an `operationStrategy`. The `calculator` stores that reference in a field. Every call to `calculate(x, y)` is forwarded to `operationStrategy.doOperation(x, y)`. The calculator never knows — or cares — whether it is adding, subtracting, multiplying, or dividing. Any class that implements `operationStrategy` can be plugged in without touching `calculator` at all.

The four structural roles are:

Role	This project
Strategy (interface)	<code>OperationStrategy</code>

DivisionOperationStrategy (Concrete Strategy) — implements `operationStrategy` by returning `x / y` using integer division.

calculator (Context) — stores an `operationStrategy` reference and delegates all arithmetic to it. It is entirely decoupled from every concrete strategy class.

DesignPatternsApplication (Client / entry point) — starts the Spring Boot application and exercises the pattern by wiring `AdditionOperationStrategy` into a `Calculator` and invoking `calculate(5, 20)`.

Code walkthrough — every line explained

`OperationStrategy.java`

```
package com.design.patterns.strategy; public interface OperationStrategy { public int
doOperation(int x, int y); }
```

Line by line:

`package com.design.patterns.strategy;` — Declares that this compilation unit belongs to the `com.design.patterns.strategy` package. It is placed in its own package (separate from the concrete implementations which live in `.concretStrategy` and the context which lives in `.context`) so that the strategy interface layer is isolated: any code that depends only on the abstraction imports from this package and never needs to know the concrete sub-package exists.

(blank line) — Separates the package declaration from the type declaration, following standard Java style.

`public interface OperationStrategy {` — Declares a public interface named `operationStrategy`. Using an `interface` rather than an abstract class is the canonical choice for the Strategy pattern: it enforces zero shared state, allows implementing classes to freely extend any superclass they choose, and makes the contract maximally explicit with no hidden behaviour. The opening `{` begins the interface body.

`public int doOperation(int x, int y);` — Declares the single abstract method that every concrete strategy must implement. `public` is redundant on an interface method (all interface methods are implicitly public) but is written explicitly here for clarity. It returns `int` so that the caller receives the arithmetic result directly without wrapper types or output parameters. The parameters `x` and `y` are named generically to keep the interface neutral — it does not imply addition, subtraction, or any specific operation. The `;` ends the method declaration without a

body, as is required for abstract interface methods in Java 8 (the version targeted by this project's parent POM).

(blank line) — Trailing blank line inside the interface body before the closing brace.

} — Closes the `operationStrategy` interface body.

`AdditionOperationStrategy.java`

```
package com.design.patterns.strategy.concretStrategy; import
com.design.patterns.strategy.OperationStrategy; public class AdditionOperationStrategy
implements OperationStrategy { @Override public int doOperation(int x, int y) { return x +
y; } }
```

Line by line:

`package com.design.patterns.strategy.concretStrategy;` — Places this class in the `concretStrategy` sub-package. Grouping all four concrete strategies here separates "what algorithms exist" from "what the algorithm contract is" and from "who uses the algorithm". This mirrors a clean layering principle: strategy interface, concrete strategies, and context each occupy their own package.

(blank line) — Separates the package declaration from the import block, per Java convention.

`import com.design.patterns.strategy.OperationStrategy;` — Brings the `operationStrategy` interface into scope so the `implements OperationStrategy` clause compiles. It is a named single-type import rather than a wildcard (`strategy.*`) so that the dependency is explicit and readable at a glance.

(blank line) — Separates the import block from the class declaration.

`public class AdditionOperationStrategy implements OperationStrategy {` — Declares a public class that fulfils the `operationStrategy` contract. `implements OperationStrategy` is the mechanism that makes this class a drop-in replacement for any other strategy wherever an `operationStrategy` reference is required. The opening `{` begins the class body.

(blank line) — Blank line inside the class for readability, separating the class header from the method.

`@Override` — Annotation that tells the compiler this method must match a signature declared in a supertype. If the signature ever drifts from `operationStrategy.doOperation(int, int)` — for example, if the interface is refactored — the compiler reports an error rather than silently creating an unrelated new method. This is a correctness guard with no runtime cost.

`public int doOperation(int x, int y) {` — Concrete implementation of the strategy method. `public` is required because the interface method is public and Java does not allow reducing visibility when overriding. `int` matches the interface return type exactly. The opening `{` begins the method body.

`return x + y;` — The addition algorithm. Returns the integer sum of the two operands. This is the only line that distinguishes `AdditionOperationStrategy` from the three other concrete strategies — the entire point of the Strategy pattern is to isolate exactly this algorithmic difference into one place.

`}` — Closes the `doOperation` method body.

`}` — Closes the `AdditionOperationStrategy` class body.

SubtractionOperationStrategy.java

```
package com.design.patterns.strategy.concretStrategy; import
com.design.patterns.strategy.OperationStrategy; public class SubtractionOperationStrategy
implements OperationStrategy { @Override public int doOperation(int x, int y) { return x -
y; } }
```

Line by line:

`package com.design.patterns.strategy.concretStrategy;` — Same `concretStrategy` sub-package as all concrete strategies, grouping them together for the cohesion reason explained above.

(blank line) — Standard Java separator between package declaration and imports.

`import com.design.patterns.strategy.OperationStrategy;` — Brings the `OperationStrategy` interface into scope so the `implements` clause compiles.

(blank line) — Separates the import block from the class declaration.

`public class SubtractionOperationStrategy implements OperationStrategy {` — Declares the subtraction concrete strategy. The class name encodes both the operation (`Subtraction`) and the role (`OperationStrategy`), making its purpose immediately readable in any code that instantiates it. The opening `{` begins the class body. *(Note: the canonical English spelling is "Subtraction" with one s; the name here preserves the source as written.)*

(blank line) — Blank line for readability inside the class body.

`@Override` — Compiler-enforced override guard, same purpose as in `AdditionOperationStrategy`.

`public int doOperation(int x, int y) {` — Concrete implementation with the same visibility and return type as the interface declaration. Opening `{` begins the method body.

`return x - y;` — The subtraction algorithm. Returns `x` minus `y`. The order of operands matters: `x - y` not `y - x`. The caller provides operands in a meaningful order and this implementation honours that order consistently with the standard mathematical convention for subtraction.

`}` — Closes the `doOperation` method body.

`}` — Closes the `SubtractionOperationStrategy` class body.

`MultiplicationOperationStrategy.java`

```
package com.design.patterns.strategy.concretStrategy; import
com.design.patterns.strategy.OperationStrategy; public class
MultiplicationOperationStrategy implements OperationStrategy { @Override public int
doOperation(int x, int y) { return x * y; } }
```

Line by line:

`package com.design.patterns.strategy.concretStrategy;` — Same concrete-strategy sub-package, same cohesion reasoning as the other concrete strategies.

(blank line) — Standard Java separator.

`import com.design.patterns.strategy.OperationStrategy;` — Pulls the strategy interface type into this compilation unit's scope so the `implements` clause is resolvable.

(blank line) — Separates imports from the class declaration.

`public class MultiplicationOperationStrategy implements OperationStrategy {` — Declares the multiplication concrete strategy. `implements OperationStrategy` binds it to the same contract as all other strategies, making it substitutable anywhere a `calculator` (or any other context) expects an `OperationStrategy` reference. The opening `{` begins the class body.

(blank line) — Whitespace for readability.

`@Override` — Compiler-enforced override guard ensuring the method signature stays consistent with the interface.

`public int doOperation(int x, int y) {` — Concrete implementation with the signature matching the interface. Opening `{` begins the method body.

`return x * y;` — The multiplication algorithm. Returns the integer product of the two operands. Like all other strategies the entire algorithm is one expression, demonstrating how cleanly the Strategy pattern isolates each variant into a single focused line.

`}` — Closes the `doOperation` method body.

`}` — Closes the `MultiplicationOperationStrategy` class body.

`DivisionOperationStrategy.java`

```
package com.design.patterns.strategy.concretStrategy; import
com.design.patterns.strategy.OperationStrategy; public class DivisionOperationStrategy
implements OperationStrategy { @Override public int doOperation(int x, int y) { return x /
y; } }
```

Line by line:

`package com.design.patterns.strategy.concretStrategy;` — Same concrete-strategy sub-package as the three other concrete strategies.

(blank line) — Standard Java separator between package declaration and import block.

`import com.design.patterns.strategy.OperationStrategy;` — Makes `operationStrategy` visible in this compilation unit so the `implements` clause resolves correctly.

(blank line) — Separates the import block from the class declaration.

`public class DivisionOperationStrategy implements OperationStrategy {` — Declares the division concrete strategy. It follows the identical structural pattern as every other concrete strategy in this project. The opening `{` begins the class body.

(blank line) — Whitespace for readability inside the class.

`@Override` — Ensures the method truly overrides the interface method and that any future signature mismatch is caught at compile time.

`public int doOperation(int x, int y) {` — Concrete implementation. Same visibility and return type as the interface. Opening `{` begins the method body.

`return x / y;` — The integer division algorithm. Returns the quotient of `x` divided by `y` using Java's integer division, which truncates any fractional part toward zero. This implementation

does not guard against `y == 0`; that is a deliberate simplification in this educational example to keep attention on the pattern structure rather than defensive programming edge cases.

} — Closes the `doOperation` method body.

} — Closes the `DivisionOperationStrategy` class body.

Calculator.java

```
package com.design.patterns.context; import
com.design.patterns.strategy.OperationStrategy; public class Calculator { private
OperationStrategy operationStrategy; public Calculator(OperationStrategy
_operationStrategy) { this.operationStrategy = _operationStrategy; } public int
calculate(int x, int y) { return operationStrategy.doOperation(x, y); } }
```

Line by line:

`package com.design.patterns.context;` — Places the `calculator` class in the `context` sub-package. Naming the package `context` directly mirrors GoF terminology: the *context* is the object that holds a strategy reference and delegates work to it. Keeping it in its own package reinforces the separation between the context role and the strategy role — any code can import the context without importing the strategies, and vice versa.

(blank line) — Standard Java separator between the package declaration and the import block.

`import com.design.patterns.strategy.OperationStrategy;` — Brings the `operationStrategy` interface type into scope. The context depends only on the interface, not on any concrete strategy class. This is the core of the Dependency Inversion Principle at work: `calculator` imports from the `strategy` package (the abstraction), never from `strategy.concretStrategy` (the implementations). This import is the only cross-package dependency `calculator` has.

(blank line) — Separates the import block from the class declaration.

`public class Calculator {` — Declares the context class. It is `public` so that the client (`DesignPatternsApplication`) in a different package can instantiate it. Opening `{` begins the class body.

(blank line) — Blank line separating the class header from the field declaration, following standard Java formatting.

`private OperationStrategy operationStrategy;` — Declares an instance field that holds a reference to whichever strategy this calculator is currently configured with. It is typed as the *interface* (`operationStrategy`), not as any concrete class — this is the polymorphic dependency

that makes the Strategy pattern work. It is `private` to enforce encapsulation: no external code can read or overwrite the strategy directly; it can only be set via the constructor. This field is the single point through which the entire pattern's flexibility flows.

(blank line) — Blank line separating the field declaration from the constructor.

`public Calculator(OperationStrategy _operationStrategy) {` — Declares the constructor that accepts an `operationStrategy`. Using constructor injection (rather than a setter or a default instantiation inside the constructor body) makes the dependency mandatory and visible: you cannot create a `calculator` without supplying a strategy, preventing the field from ever being `null` due to a forgotten initialisation. The parameter is prefixed with `_` to distinguish it from the field of the same base name — a naming convention chosen here to avoid ambiguity, though `this.operationStrategy = operationStrategy` with identical names is equally valid. The opening `{` begins the constructor body.

`this.operationStrategy = _operationStrategy;` — Stores the injected strategy into the instance field. `this.operationStrategy` explicitly refers to the field declared above; `_operationStrategy` is the constructor parameter. This single assignment is the entire body of the constructor: all the context setup amounts to wiring the strategy reference.

`}` — Closes the constructor body.

(blank line) — Blank line separating the constructor from the `calculate` method.

`public int calculate(int x, int y) {` — Declares the public API of the context. Callers ask the `calculator` to compute a result; they pass two integer operands and receive the result. The method name `calculate` is intentionally generic — it does not say "add" or "multiply" because the actual operation depends on which strategy is held. This genericity is important: the same method name works regardless of which algorithm is plugged in. Opening `{` begins the method body.

`return operationStrategy.doOperation(x, y);` — Delegates the computation entirely to the strategy. The `calculator` performs no arithmetic itself; it simply forwards the call. This single line is the heart of the Strategy pattern in the context: the context knows *that* an operation must happen but not *how* it happens. The result returned by the strategy is passed straight back to the caller without transformation.

`}` — Closes the `calculate` method body.

(blank line) — Trailing blank line before the class closing brace.

`}` — Closes the `calculator` class body.

DesignPatternsApplication.java

```
package com.design.patterns; import org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication; import
com.design.patterns.context.Calculator; import
com.design.patterns.strategy.concretStrategy.AdditionOperationStrategy;
@SpringBootApplication public class DesignPatternsApplication { public static void
main(String[] args) { SpringApplication.run(DesignPatternsApplication.class, args);
Calculator calculator = new Calculator(new AdditionOperationStrategy()); int calculate =
calculator.calculate(5, 20); System.out.println(calculate); } }
```

Line by line:

`package com.design.patterns;` — Declares that this class belongs to the root package `com.design.patterns`. Spring Boot's default component-scan starts from the package of the `@SpringBootApplication`-annotated class, so placing the entry point at the root package ensures all sub-packages (`context`, `strategy`, `strategy.concretStrategy`) are automatically scanned for Spring components.

(blank line) — Standard separator between the package declaration and the import block.

`import org.springframework.boot.SpringApplication;` — Brings the `SpringApplication` utility class into scope. `SpringApplication.run()` is the standard Spring Boot bootstrap method; without this import the call on the corresponding line would not compile.

`import org.springframework.boot.autoconfigure.SpringBootApplication;` — Brings the `@SpringBootApplication` meta-annotation into scope. Without this import the annotation on the class declaration would be unresolvable at compile time.

(blank line) — Blank line between the framework imports and the application-specific imports. Separating import groups by origin (framework vs. application) is a common Java style convention that improves readability.

`import com.design.patterns.context.Calculator;` — Brings the `calculator` context class into scope so it can be instantiated inside `main()`.

`import com.design.patterns.strategy.concretStrategy.AdditionOperationStrategy;` — Brings the concrete addition strategy into scope. This is the only place in the entire project where a concrete strategy class is referenced by name from outside its own file. Everywhere else, the code depends only on the `operationStrategy` interface. This is intentional: the client is precisely the one place where a specific algorithm is chosen, so the concrete type name surfaces only here.

(blank line) — Separates the import block from the class declaration.

`@SpringBootApplication` — A Spring Boot meta-annotation that is shorthand for three annotations combined: `@Configuration` (this class can declare Spring beans), `@EnableAutoConfiguration` (Spring Boot automatically configures the application context based on the classpath), and `@ComponentScan` (Spring scans the `com.design.patterns` package and all sub-packages for annotated components). Without this annotation, the Spring context would not be bootstrapped. It is placed on the entry-point class because Spring Boot uses that class's package as the scan root.

`public class DesignPatternsApplication {` — Declares the Spring Boot application class. It is `public` so that the JVM can invoke its `main` method as the application entry point. Opening `{` begins the class body.

(blank line) — Blank line separating the class header from the method declaration.

`public static void main(String[] args) {` — The standard Java entry-point method signature. `public` is required so the JVM can call it from outside the class. `static` is required so the JVM can invoke it without needing a pre-existing instance of the class. `void` is required because the JVM ignores any return value from `main`. `String[] args` receives command-line arguments passed when the JVM is launched; Spring Boot uses these to allow property overrides. The opening `{` begins the method body.

`SpringApplication.run(DesignPatternsApplication.class, args);` — Starts the Spring Boot application. `DesignPatternsApplication.class` is the primary source class: Spring Boot uses its package as the component-scan root and its `@SpringBootApplication` annotation to drive auto-configuration. `args` passes command-line arguments through to Spring, allowing properties such as `server.port` to be overridden at launch. This line starts an embedded Tomcat server (because `spring-boot-starter-web` is on the classpath), initialises the application context, and writes the Spring Boot ASCII banner plus INFO startup log lines to standard output. After this call returns, the embedded server continues running on background threads.

(blank line) — Blank line separating the Spring Boot bootstrap call from the pattern demonstration code. This visual separation makes clear that the lines below are the pattern demo, independent of the Spring context.

`Calculator calculator = new Calculator(new AdditionOperationStrategy());` — Demonstrates the Strategy pattern in its simplest form. `new AdditionOperationStrategy()` creates a concrete strategy instance (an object with no fields). That instance is immediately passed into `new calculator(...)` via constructor injection. The `calculator` local variable holds the fully configured context object. This is the moment where the strategy *choice* is made at runtime: to switch to multiplication, one would write `new MultiplicationOperationStrategy()` here and nothing else in the codebase would change.

`int calculate = calculator.calculate(5, 20);` — Invokes the context's generic `calculate` API with operands `5` and `20`. The local variable `calculate` (a verb used as a noun, mirroring the method name) stores the returned `int` result. Internally, this chains to `operationStrategy.doOperation(5, 20)` inside `calculator`, which dynamic-dispatches to `AdditionOperationStrategy.doOperation(5, 20)`, which evaluates `5 + 20` and returns `25`. So `calculate` now holds the value `25`.

`System.out.println(calculate);` — Prints the result to standard output followed by a newline. `System.out` is the JVM's standard output `PrintStream`. `println` converts the primitive `int` value `25` to the string `"25"` and writes it with a trailing platform-dependent line separator. This is the observable output of the entire pattern demonstration.

(blank line) — Trailing blank line before the method closing brace.

`}` — Closes the `main` method body.

(blank line) — Blank line before the class closing brace.

`}` — Closes the `DesignPatternsApplication` class body.

Why these design decisions

Why an interface, not an abstract class?

An interface enforces a pure contract with no shared state or partial implementation. Any class — regardless of its own inheritance hierarchy — can implement `operationStrategy`. If an abstract class had been used, every concrete strategy would be forced to extend it, consuming Java's single inheritance slot and coupling all strategies to a common ancestor that adds no real value.

Why constructor injection in `calculator`?

Constructor injection makes the dependency mandatory and visible at compile time. You cannot instantiate a `calculator` without supplying a strategy. This prevents null-pointer errors from forgotten setter calls and makes the dependency explicit in the public API. The alternative — a `setStrategy()` setter — would allow a `calculator` to exist with no strategy, which is an invalid state.

Why does `calculator` depend on `operationStrategy` and not on a specific class?

This is the Dependency Inversion Principle in action. `calculator` depends on an abstraction, not a concretion. This means `calculator` is closed for modification and open for extension: adding a new operation (e.g., `ModuloOperationStrategy`) requires writing one new class and changing zero existing classes — the Open/Closed Principle.

Why are the concrete strategies in a sub-package (`concretStrategy`) separate from the interface?

The interface (`OperationStrategy`) is what the context and any other consumer depend on. The concrete implementations are interchangeable details. Placing them in a sub-package makes the dependency direction visible in the package structure: `context` imports `strategy` (the abstraction), never `strategy.concretStrategy` (the concretions).

Why does `main()` call `SpringApplication.run()` if the pattern demo does not use Spring?

This project scaffolds the Spring Boot application runner as a standard container so the pattern can later be wired as a real Spring service (for example, injecting strategies as `@Component` beans selected by a `@Qualifier`). The pattern demonstration code that follows the `run()` call intentionally uses plain `new` instantiation to keep the core mechanism visible and free of framework detail.

Trade-offs:

Trade-off	Impact
Adding a new operation requires a new class file	More files than a single <code>if/switch</code> block, but each is tiny, independently testable, and adding one never touches existing code
The context always needs a strategy injected	Slightly more setup for the caller, but eliminates invalid-null-state bugs at compile time
Integer division truncates silently	Acceptable for a pedagogical example; production code would use <code>double</code> or <code>BigDecimal</code> and guard against division by zero
<code>OperationStrategy</code> is a functional interface	Can be replaced by a lambda <code>((x, y) -> x + y)</code> , which is a strength — lambdas are lightweight strategies

Execution flow (step-by-step trace of what happens when `main()` runs)

1. The JVM invokes `DesignPatternsApplication.main(args)`.
2. `SpringApplication.run(DesignPatternsApplication.class, args)` is called.
3. Spring Boot initialises an `AnnotationConfigServletWebServerApplicationContext`.
4. The embedded Tomcat web server starts on port 8080 (default).
5. Spring Boot writes its ASCII-art banner and INFO startup log lines to standard output.

6. The log line `Started DesignPatternsApplication in X.XXX seconds (JVM running for X.XXX)` appears.
7. `run()` returns after the context is fully refreshed; the Tomcat server remains alive on non-daemon background threads.
8. Execution resumes in `main()` after the `run()` call.
9. `new AdditionOperationStrategy()` constructs an instance of the addition strategy. It is a plain Java object with no fields and no initialisation logic.
10. `new Calculator(additionStrategy)` constructs a `Calculator`. Inside the constructor body, `this.operationStrategy = _operationStrategy` stores the `AdditionOperationStrategy` reference into the `operationStrategy` field.
11. `calculator.calculate(5, 20)` is invoked.
12. Inside `calculate`, `operationStrategy.doOperation(5, 20)` is called on the field.
13. The JVM uses dynamic dispatch: the runtime type of `operationStrategy` is `AdditionOperationStrategy`, so `AdditionOperationStrategy.doOperation` is invoked.
14. Inside `doOperation`, `5 + 20` is evaluated: result is 25.
15. `doOperation` returns 25 to `calculate`, which returns 25 to `main`.
16. The local variable `calculate` now holds the primitive value 25.
17. `System.out.println(25)` converts 25 to the string "25" and writes it followed by a newline to standard output.
18. `main()` returns; the JVM remains alive because the Spring Boot embedded Tomcat server holds live non-daemon threads.

Expected output

Spring Boot startup banner and INFO log lines appear first (exact content varies by Spring Boot version and system timing). The line produced by the pattern demonstration itself follows immediately after the startup sequence:

```
. ____ _ _ _ _ / \ / _ _ ' _ _ _ _ ( _ ) _ _ _ _ \ \ \ \ ( ( ) \ _ _ | ' _ | ' _ | ' _ \ / _ ' | \
\ \ \ \ \ / _ _ ) | | _ | | | | | | ( _ | | ) ) ) ' | _ _ | . _ | | | | | _ \ , | / / / /
=====|_|=====|_|/=//_/_/_/ :: Spring Boot :: (v2.7.7) ... INFO
o.s.boot.SpringApplication - Starting DesignPatternsApplication ... ... INFO
o.s.boot.SpringApplication - Started DesignPatternsApplication in X.XXX seconds (JVM
running for X.XXX) ... 25
```

The key line produced by the pattern demonstration is:

This is the result of `AdditionOperationStrategy.doOperation(5, 20) → 5 + 20 → 25`, printed by `System.out.println(calculate)`.

How to run

From the module directory (`Behavioral/Strategy_Design_Pattern/`):

```
# Build the fat JAR mvn -o clean package -DskipTests # Run via the Spring Boot fat JAR
java -jar target/Strategy_Design_Pattern-0.0.1-SNAPSHOT.jar
```

From the repository root (builds the module and its parent POM):

```
mvn -o clean package -pl Behavioral/Strategy_Design_Pattern -am -DskipTests java -jar
Behavioral/Strategy_Design_Pattern/target/Strategy_Design_Pattern-0.0.1-SNAPSHOT.jar
```

Main class fully qualified name (for reference):

```
com.design.patterns.DesignPatternsApplication
```

Other real-world problems this pattern can solve

Domain	Context class	Interchangeable strategies
Payment processing	PaymentProcessor	CreditCardStrategy, PayPalStrategy, CryptoStrategy
File compression	Archiver	ZipStrategy, GzipStrategy, Bzip2Strategy
Sorting	DataSorter	QuickSortStrategy, MergeSortStrategy, TimSortStrategy

Authentication	AuthService	JwtStrategy, OAuth2Strategy, LdapStrategy
Route navigation	Navigator	FastestRouteStrategy, ShortestRouteStrategy, EcoRouteStrategy
Logging output	Logger	ConsoleLogStrategy, FileLogStrategy, RemoteSyslogStrategy
Tax calculation	Invoice	USTaxStrategy, EU VatStrategy, TaxExemptStrategy
Pricing discounts	PriceCalculator	SeasonalDiscountStrategy, LoyaltyDiscountStrategy, NoDiscountStrategy
Text rendering	DocumentRenderer	HtmlRenderStrategy, PdfRenderStrategy, PlainTextStrategy

In every case the context class remains unchanged; only the strategy passed in at construction (or injected by a DI container such as Spring) determines the runtime behaviour.